In [1]:
```python
import numpy as np
import pandas as pd
```

In [2]:
```python
pd.__version__
```

Out[2]: '2.3.3'

pandas :

1. Pandas in Python is an open-source library used for data manipulation, analysis, and cleaning.

It provides fast, flexible, and expressive data structures to work with structured data (like tables in spreadsheets or SQL databases)

2.**Series – 1D array (like a column in Excel).**

DataFrame – 2D array (like an Excel sheet or SQL table).

pandas series :

In [3]:
```python
l = [10,20,30,40]    # series using list

s= pd.Series(l)
print(s)
print(s.shape)
type(s)
```

```
0    10
1    20
2    30
3    40
dtype: int64
(4,)
```

Out[3]: pandas.core.series.Series

In [4]:
```
l = [10,20,30,40]    # changing index name

s= pd.Series(l, index=['a','b','c','d'])
print(s)
```

```
a    10
b    20
c    30
d    40
dtype: int64
```

In [5]:
```
 # series using 1D array

a= np.array([10,20,30,40])        # can be change index name
s = pd.Series(a)

print(s)
print(s.shape)
type(s)
```

```
0    10
1    20
2    30
3    40
dtype: int64
(4,)
```

Out[5]:
```
pandas.core.series.Series
```

In [6]:
```
#series using dictionary
d={'a':10,'b':20,'c':30,'d':40}
s= pd.Series(d)
s
```

Out[6]:
```
a    10
b    20
c    30
d    40
dtype: int64
```

pandas Data frame :

In [7]:
```
# series using dictionary
d={'a':[10,11],'b':[20,21],'c':[30,31],'d':[40,41]}
s= pd.Series(d)
s
```

Out[7]:
```
a    [10, 11]
b    [20, 21]
c    [30, 31]
d    [40, 41]
dtype: object
```

In [8]:
```
# Data frame using dictionary
d={'a':[10,11],'b':[20,21],'c':[30,31],'d':[40,41]}   # in Data frame when we us
s= pd.DataFrame(d)
s
```

Out[8]:

	a	b	c	d
0	10	20	30	40
1	11	21	31	41

In [9]:
```
type(s)
```

Out[9]:
```
pandas.core.frame.DataFrame
```

In [10]:
```
s.shape
```

Out[10]:
```
(2, 4)
```

In [11]:
```
# creating Data Frame using 2D array:
nl= [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
df= pd.DataFrame(nl)
df
```

Out[11]:

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

In [12]:
```
# editing row and colounm name:
nl= [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
df= pd.DataFrame(nl,index=['a','b','c'],columns=['i','j','k','l'])
df
```

Out[12]:

	i	j	k	l
a	1	2	3	4
b	5	6	7	8
c	9	10	11	12

Pandas:

In [13]:
```
d = {'Age' :[22,25,35,40,45,24],
     'Salary':[11000,12000,13000,14000,15000,16000],
      'Gender':['F','F','F','M','M','F'],
      'address':['korba','bsp','Bkp','surajpur','dipka',' raipur']

    }
df = pd.DataFrame(d,index=['a','b','c','d','e','f'])
df
```

Out[13]:

	Age	Salary	Gender	address
a	22	11000	F	korba
b	25	12000	F	bsp
c	35	13000	F	Bkp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [14]:

```
df1= df.set_index('Age')
df1
```

Out[14]:

	Salary	Gender	address
Age			
22	11000	F	korba
25	12000	F	bsp
35	13000	F	Bkp
40	14000	M	surajpur
45	15000	M	dipka
24	16000	F	raipur

In [15]:

```
df
```

Out[15]:

	Age	Salary	Gender	address
a	22	11000	F	korba
b	25	12000	F	bsp
c	35	13000	F	Bkp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [16]:

```
#droping single row,multiple row
df2 =df.drop(index='c')
df2
```

Out[16]:

	Age	Salary	Gender	address
a	22	11000	F	korba
b	25	12000	F	bsp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [17]:
```
df2 =df.drop(index=['c','a'])
df2
```

Out[17]:

	Age	Salary	Gender	address
b	25	12000	F	bsp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [18]:
```
df2 =df.drop(columns=['Gender','Salary'])
df2
```

Out[18]:

	Age	address
a	22	korba
b	25	bsp
c	35	Bkp
d	40	surajpur
e	45	dipka
f	24	raipur

sorting

In [19]:
```
df
```

Out[19]:

	Age	Salary	Gender	address
a	22	11000	F	korba
b	25	12000	F	bsp
c	35	13000	F	Bkp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [20]:

```
df2=df.sort_values(by= 'Age',ascending= True)
df2
```

Out[20]:

	Age	Salary	Gender	address
a	22	11000	F	korba
f	24	16000	F	raipur
b	25	12000	F	bsp
c	35	13000	F	Bkp
d	40	14000	M	surajpur
e	45	15000	M	dipka

In [21]:

```
df2=df.sort_values(by= 'Salary',ascending= True)
df2
```

Out[21]:

	Age	Salary	Gender	address
a	22	11000	F	korba
b	25	12000	F	bsp
c	35	13000	F	Bkp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [22]:

```
df2=df.sort_values(by= 'Age',ascending= False)
df2
```

Out[22]:

	Age	Salary	Gender	address
e	45	15000	M	dipka
d	40	14000	M	surajpur
c	35	13000	F	Bkp
b	25	12000	F	bsp
f	24	16000	F	raipur
a	22	11000	F	korba

In [23]:
```
df
```

Out[23]:

	Age	Salary	Gender	address
a	22	11000	F	korba
b	25	12000	F	bsp
c	35	13000	F	Bkp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [24]:
```
df['Aadhar'] = ['N','Y','Y','N','N','Y']  #adding new colunm
df
```

Out[24]:

	Age	Salary	Gender	address	Aadhar
a	22	11000	F	korba	N
b	25	12000	F	bsp	Y
c	35	13000	F	Bkp	Y
d	40	14000	M	surajpur	N
e	45	15000	M	dipka	N
f	24	16000	F	raipur	Y

In [25]:
```
df1= {'city':['mumbai','delhi','banglor','chennai'],
      'Temp':[68,65,75,70]}
df1=pd.DataFrame(df1)
df1
```

Out[25]:

	city	Temp
0	mumbai	68
1	delhi	65
2	banglor	75
3	chennai	70

In [26]:

```
df2= {'city':['mumbai','delhi','banglor','hydrabad'],
      'humidity':[30,29,40,35]}
df2=pd.DataFrame(df2)
df2
```

Out[26]:

	city	humidity
0	mumbai	30
1	delhi	29
2	banglor	40
3	hydrabad	35

In [27]:

```
#concatenation
# colunm=1,row=0 (pandas)

pd.concat([df1,df2],axis=1)
```

Out[27]:

	city	Temp	city	humidity
0	mumbai	68	mumbai	30
1	delhi	65	delhi	29
2	banglor	75	banglor	40
3	chennai	70	hydrabad	35

In [28]:

```
df_new =pd.concat([df1,df2],axis=0)
df_new
```

Out[28]:

	city	Temp	humidity
0	mumbai	68.0	NaN
1	delhi	65.0	NaN
2	banglor	75.0	NaN
3	chennai	70.0	NaN
0	mumbai	NaN	30.0
1	delhi	NaN	29.0
2	banglor	NaN	40.0
3	hydrabad	NaN	35.0

In [29]:

```
df_new =pd.concat([df1,df2],ignore_index= True)
df_new
```

Out[29]:

	city	Temp	humidity
0	mumbai	68.0	NaN
1	delhi	65.0	NaN
2	banglor	75.0	NaN
3	chennai	70.0	NaN
4	mumbai	NaN	30.0
5	delhi	NaN	29.0
6	banglor	NaN	40.0
7	hydrabad	NaN	35.0

In [30]:

```
left_df= pd.merge(df1,df2, on ='city',how='left')
left_df
```

Out[30]:

	city	Temp	humidity
0	mumbai	68	30.0
1	delhi	65	29.0
2	banglor	75	40.0
3	chennai	70	NaN

In [31]:

```
right_df= pd.merge(df1,df2, on ='city',how='right')
right_df
```

Out[31]:

	city	Temp	humidity
0	mumbai	68.0	30
1	delhi	65.0	29
2	banglor	75.0	40
3	hydrabad	NaN	35

In [32]:
```
# inner join = common data record
```

In [33]:
```
inner_df= pd.merge(df1,df2, on ='city',how='inner')
inner_df
```

Out[33]:

	city	Temp	humidity
0	mumbai	68	30
1	delhi	65	29
2	banglor	75	40

In [34]:
```
#outer join = all the record from both data frame
```

In [35]:
```
outer_df= pd.merge(df1,df2, on ='city',how='outer')
outer_df
```

Out[35]:

	city	Temp	humidity
0	banglor	75.0	40.0
1	chennai	70.0	NaN
2	delhi	65.0	29.0
3	hydrabad	NaN	35.0
4	mumbai	68.0	30.0

loc and iloc

loc=index name

iloc= index number

[row , colunm]

In [36]:
```
df
```

Out[36]:

	Age	Salary	Gender	address	Aadhar
a	22	11000	F	korba	N
b	25	12000	F	bsp	Y
c	35	13000	F	Bkp	Y
d	40	14000	M	surajpur	N
e	45	15000	M	dipka	N
f	24	16000	F	raipur	Y

In [58]:
```
df.loc['a':'c',:]  #[start from a upto c row , all colunm]
```

Out[58]:

	Age	Salary	Gender	address	Aadhar
a	22	11000	F	korba	N
b	25	12000	F	bsp	Y
c	35	13000	F	Bkp	Y

In [56]:
```
df.iloc[1:3, [0,1]]   #imp point= multiple row and multiple colunm no need brack
                        # ex: iloc[0:4],[1,2]] this will give error
                              # iloc[0:4],[1,2] this will give right output
                      # above imp point is applicable for both loc,iloc
```

Out[56]:

	Age	Salary
b	25	12000
c	35	13000

In [65]:
```
df.iloc[:, 0:4]
```

Out[65]:

	Age	Salary	Gender	address
a	22	11000	F	korba
b	25	12000	F	bsp
c	35	13000	F	Bkp
d	40	14000	M	surajpur
e	45	15000	M	dipka
f	24	16000	F	raipur

In [ ]:

In [ ]: